Technische Universität Braunschweig

Master Thesis
In the Course of Studies Mathematics (M.Sc.)
At the Institut für Mathematische Optimierung

A Branch- and Bound Method for Bilevel Parallel Machine Scheduling with Adversarial Job Selection

Ole Damke
5029861
o.damke@tu-braunschweig.de

First Reviewer: Prof. Dr. Maximilian Merkert
Institut für Mathematische Optimierung
Second Reviewer: Univ.-Prof. Dr. rer. nat. habil. Christian Kirches
Institut für Mathematische Optimierung

Submission Date: March 29, 2026

Abstract

This thesis investigates a bilevel knapsack-scheduling problem where a leader and follower have conflicting objectives regarding system makespan. In the upper-level problem, the leader selects quantities of job types within a budget constraint to maximize the makespan. In the lower-level problem, the follower schedules the selected jobs on parallel identical machines to minimize the makespan. We develop a branch-and-bound algorithm that exploits the hierarchical structure of this bilevel problem. To prune, we derive two different upper bounding techniques: a knapsack-scheduling relaxation that optimally solves a continuous allocation problem, and a Max-LPT approach that provides a $\frac{3}{4}$-approximation guarantee by combining longest processing time scheduling with knapsack optimization. Computational experiments compare both bounding approaches against complete enumeration and against the MIBS solver from the literature.

Keywords: Bilevel optimization, branch-and-bound, knapsack problem, scheduling, combinatorial optimization

Contents

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Problem Overview . . . 1
1.3 Contributions . . . 1
1.4 Organization . . . 2

2 Problem Formulation **3**
2.1 Bilevel Optimization Framework . . . 3
2.2 The Knapsack-Scheduling Problem . . . 3
2.2.1 Problem Data . . . 3
2.2.2 Leader's Problem (Upper Level) . . . 3
2.2.3 Follower's Problem (Lower Level) . . . 4
2.3 Complete Bilevel Formulation . . . 4
2.4 Problem Characteristics . . . 4
2.5 Interpretation: Why Maximize Makespan? . . . 5
2.6 Literature Review . . . 5

3 Solution Methodology **7**
3.1 Complete Enumeration (Baseline) . . . 7
3.1.1 Algorithm . . . 7
3.1.2 Combinatorial Growth . . . 7
3.2 Branch-and-Bound Algorithm . . . 8
3.2.1 Overview . . . 8
3.2.2 Search Tree Structure . . . 8
3.2.3 Follower's Scheduling Algorithm . . . 8
3.2.4 Upper Bound Computation . . . 10
3.2.5 Branch-and-Bound Algorithm . . . 11
3.3 The Ceiling Bound . . . 12
3.3.1 Knapsack Scheduling Relaxation . . . 12
3.3.2 Dynamic Programming Approach . . . 13
3.3.3 Optimization: Reversed Item Order . . . 13
3.4 Another Upper Bound: Maximizing the LPT Rule . . . 14
3.4.1 The Max-LPT Problem . . . 14
3.4.2 Approximation Guarantee . . . 14
3.4.3 Early Optimality Detection via Max-LPT . . . 16
3.4.4 Ordered Makespan Notation . . . 17
3.4.5 Algorithm Description and Correctness . . . 17
3.4.6 Connection to the Knapsack-Scheduling Problem . . . 19
3.4.1 Modified Branch-and-Bound with Max-LPT Upper Bound . . . 21

4 Implementation Details **23**
4.1 Software Architecture . . . 23

4.2 Key Data Structures . . . 23
4.2.1 Problem Node . . . 23
4.3 Search Tree Structure and Algorithmic Optimizations . . . 24
4.3.1 Depth-First Search and Node Expansion . . . 24
4.3.2 Incumbent Initialization . . . 25
4.3.3 Incremental Budget and Contribution Tracking . . . 25

5 Computational Experiments 26
5.1 Experimental Setup . . . 26
5.1.1 Test Environment . . . 26
5.1.2 Performance Metrics . . . 26
5.1.3 Test by Pattern . . . 26
5.1.4 Verification Study: Algorithm Correctness . . . 28
5.2 Sensitivity Analysis: Single Parameter Scaling . . . 28
5.2.1 Methodology . . . 29
5.2.2 Scaling Number of Job Types . . . 29
5.2.3 Scaling Number of Machines . . . 29
5.2.4 Scaling Budget Multiplier . . . 31
5.3 Grid Sensitivity Analysis . . . 31
5.3.1 Motivation . . . 31
5.3.2 Methodology . . . 31
5.3.3 Results: Status Distribution by Budget Multiplier . . . 32
5.3.4 Runtime and Node Count Comparison: Ceiling vs MaxLPT . . . 32
5.3.5 Root-Optimal Case: Runtime Comparison . . . 33
5.4 Difficulty Analysis by Parameter Configuration . . . 34
5.5 Comparison with MIBS Solver . . . 35
5.5.1 MIBS Background . . . 35
5.5.2 Experimental Design . . . 35
5.5.3 Results . . . 36
5.5.4 Key Algorithm Insights and Practical Findings . . . 37
5.6 Summary . . . 37

6 Conclusion and Future Work 39
6.1 Summary and Main Contributions . . . 39
6.2 Key Results and Insights . . . 39
6.3 Limitations . . . 40
6.4 Future Directions . . . 40
6.5 Closing Remarks . . . 40

7 Statutory Declaration 44

1 Introduction

1.1 Motivation

Hierarchical decision-making is a fundamental challenge in many real-world systems. Consider a production facility that must decide which types of jobs to accept within a limited budget, knowing that a scheduler will later assign these jobs to machines to minimize completion time. The facility manager needs to plan for worst-case scenarios: what is the maximum makespan that could occur if jobs were selected adversarially to stress the system, even when the scheduler optimally minimizes completion time? Understanding this worst-case bound is crucial for capacity planning, resource provisioning, and ensuring the facility can meet service level agreements under any job selection within budget.

This scenario exemplifies a *bilevel optimization problem*, where one decision-maker (the leader) makes choices that affect another decision-maker (the follower), who then optimizes their own objective. Such problems could arise in supply chain management, network design, resource allocation, and competitive markets.

1.2 Problem Overview

We study a bilevel optimization problem combining two classic research problems of mathematical optimization:

- **Upper level (Leader):** Unbounded Knapsack problem – select jobs from a finite set of job types within a budget
- **Lower level (Follower):** Scheduling problem on parallel identical machines – assign jobs to machines to minimize makespan

The leader's objective is to maximize the makespan (completion time). This models a worst-case robust optimization approach, where the leader tries to account for the worst possible makespan, even when the follower actually minimizes completion time by finding an optimal schedule. The lower-level scheduling problem is solved via integer programming using Gurobi [Gur26].

1.3 Contributions

Throughout this thesis, we refer to this problem as the *bilevel knapsack-scheduling problem*, which concisely describes the combination of knapsack-based job selection at the upper level and parallel machine scheduling at the lower level with adversarial job selection.

This thesis makes the following contributions:

1. Formal mathematical model of the bilevel knapsack-scheduling problem
2. Branch-and-bound algorithm with two different bounds to prune subtrees based on a knapsack scheduling relaxation

3. Efficient dynamic programming solver for the two bounding subproblems
4. Comprehensive computational study demonstrating practical scalability comparing the two different approaches with each other and with a solver from the literature

1.4 Organization

The remainder of this thesis is organized as follows. Section 2 presents the mathematical formulation of the bilevel problem. Section 3 describes the branch-and-bound algorithm and bounding techniques. Section 4 discusses implementation details. Section 5 reports computational results. Section 6 concludes with a summary and future research directions.

2 Problem Formulation

2.1 Bilevel Optimization Framework

A bilevel optimization problem has a hierarchical structure with two decision-makers.

Definition 2.1.1 (Bilevel Optimization Problem)**.**

$$\max_x \quad F(x, y^*(x)) \tag{2.1}$$

$$\text{s.t.} \quad x \in X$$

$$\text{where } y^*(x) \in \arg\min_y \{f(x, y) : y \in Y(x)\} \tag{2.2}$$

The leader chooses x to optimize their objective F, anticipating that the follower will respond by solving their own optimization problem to find $y^*(x)$. For more information, see [Kle+21] for a survey on the field of bilevel optimization, and [DZ20] for a comprehensive mathematical foundation.

Optimistic vs. Pessimistic Formulation: When the follower's problem has multiple optimal solutions, the *optimistic formulation* assumes the follower selects the solution most favorable to the leader, while the *pessimistic formulation* assumes the least favorable choice. The above formulation implies the optimistic case.

2.2 The Knapsack-Scheduling Problem

2.2.1 Problem Data

- n job types, indexed by $i = 1, \ldots, n$
- Job type i has:
  - Processing time (duration): $d_i > 0$
  - Cost (price): $p_i > 0$
- m identical parallel machines
- Total budget: $B > 0$

2.2.2 Leader's Problem (Upper Level)

The leader selects how many jobs of each type to purchase:

Decision variables: $x_i \in \mathbb{Z}_+$ = number of jobs of type i to select

Constraints:

$$\sum_{i=1}^{n} p_i x_i \leq B \quad \text{(budget constraint)} \tag{2.3}$$

Objective: Maximize the makespan resulting from optimal scheduling:

$$\max_{x} \quad C^*_{\max}(x) \tag{2.4}$$

where $C^*_{\max}(x)$ is the optimal makespan for the follower's problem given selection x.

2.2.3 Follower's Problem (Lower Level)

Given the leader's selection $x = (x_1, \ldots, x_n)$, the follower has a total of $\sum_{i=1}^{n} x_i$ jobs to schedule.

Decision variables: For each job j, assign it to some machine

Objective: Minimize makespan (maximum machine load)

$$C^*_{\max}(x) = \min \max_{k=1,\ldots,m} L_k \tag{2.5}$$

where L_k denotes the load on machine k, defined as the total processing time of all jobs assigned to machine k after scheduling.

2.3 Complete Bilevel Formulation

We present a mixed-integer linear programming (MILP) formulation compatible with standard bilevel solvers such as MibS [Fis+16]. The formulation uses a shared makespan variable $C_{\max}$ that appears in both levels:

$$\max_{x, C_{\max}} \quad C_{\max} \tag{2.6}$$

$$\text{s.t.} \quad \sum_{i=1}^{n} p_i x_i \leq B \tag{2.7}$$

$$x_i \in \mathbb{Z}_+ \quad \forall i \in \{1, \ldots, n\}$$

$$\text{where } C_{\max} \in \quad \arg\min_{y, C_{\max}} \quad C_{\max} \tag{2.8}$$

$$\text{s.t.} \quad \sum_{k=1}^{m} y_{ik} = x_i \quad \forall i \in \{1, \ldots, n\} \tag{2.9}$$

$$C_{\max} \geq \sum_{i=1}^{n} d_i \cdot y_{ik} \quad \forall k \in \{1, \ldots, m\} \tag{2.10}$$

$$y_{ik} \in \mathbb{Z}_+ \quad \forall i, k$$

The leader's variables are x_i (number of jobs of type i to select) and the follower's variables are y_{ik} (number of jobs of type i assigned to machine k). The shared variable $C_{\max}$ represents the makespan: the leader seeks to maximize it while the follower seeks to minimize it, creating the adversarial nature of the problem.

2.4 Problem Characteristics

The computational challenge of this problem arises from multiple sources. First, the problem combines two NP-hard problems (knapsack and scheduling) in a bilevel scenario. Bilevel optimization is well-known to be computationally challenging even for simple problems

with linear objectives and constraints, making our discrete, non-convex variant particularly difficult to solve. The follower's optimal value function $C^*_{\max}(x)$ is non-convex and discontinuous, which prevents the use of gradient-based methods and complicates the analysis of the leader's objective landscape. Furthermore, the discrete nature of both levels means that standard continuous optimization techniques, such as KKT conditions, cannot be applied. Consequently, solving this problem requires combinatorial methods that explicitly explore the discrete solution space.

2.5 Interpretation: Why Maximize Makespan?

The leader maximizes makespan to identify the worst-case completion time that could arise within the budget constraint. This defensive approach models robust planning under uncertainty, where the decision-maker seeks to understand the maximum possible makespan that could result from any job selection, even when the scheduler optimizes to minimize completion time. By solving for this worst-case bound, the leader can make informed decisions about resource provisioning, capacity planning, and service level guarantees. More generally, this framework applies to any adversarial or worst-case planning scenario where a decision-maker must prepare for unfavorable conditions while an opposing agent optimizes in their own interest.

Example: A production facility manager needs to determine what workload the system must be prepared to handle. By identifying the job selection that maximizes makespan (under optimal scheduling), the manager obtains a worst-case guarantee: the system will never face a longer completion time than this bound, regardless of which specific jobs arrive within budget.

2.6 Literature Review

We provide a brief overview of selected works relevant to our problem; for a comprehensive treatment of bilevel optimization, we refer the reader to the extensive body of literature in this field.

Dempe and Zemkoho [DZ20] provide a comprehensive mathematical foundation for bilevel optimization, covering theory, optimality conditions, and solution approaches. Kleinert et al. [Kle+21] offer an extensive survey on mixed-integer programming techniques in bilevel optimization, reviewing branch-and-bound methods, cutting planes, and decomposition approaches that form the algorithmic foundation for modern bilevel solvers.

On the algorithmic side, Fischetti et al. [Fis+16] introduced intersection cuts for bilevel optimizationand and compared their results against the MibS (Mixed Integer Bilevel Solver) framework that was developed by Tahernejad, Ralphs, and DeNegre [TRD20]. MibS implements a comprehensive branch-and-cut algorithm for mixed-integer bilevel linear optimization, combining features from traditional MILP and bilevel methods with new techniques to create a flexible and robust solver. In this thesis, we compare our specialized branch-and-bound algorithm against MibS just like Fischetti et al. [Fis+16], to evaluate the benefits of problem-specific techniques versus general-purpose bilevel solvers.

Most relevant to our work, Weninger and Fukasawa [WF25] recently developed a fast combinatorial branch-and-bound algorithm for the bilevel knapsack problem with interdiction constraints. Their approach uses dynamic programming to compute strong lower bounds by solving a relaxed 2n-level version of the problem where items are processed in an online fashion. This technique shares similarities with our method, as both employ branch-and-bound combined with dynamic programming to exploit problem structure. However, their

lower-level problem is a second knapsack optimization problem, whereas ours involves parallel machine scheduling, requiring different bounding techniques.

Leitner et al. [Lei+23] further advanced the field by developing exact methods for three-level optimization problems (fortification games), demonstrating that problem-specific reformulations and specialized cuts can significantly outperform general approaches, solving instances with over 330,000 nodes to optimality.

While the general bilevel optimization literature is extensive, our specific problem combining knapsack constraints in the upper level with parallel machine scheduling in the lower level represents a novel variant. The hierarchical structure with adversarial objectives (maximize vs. minimize makespan) distinguishes our problem from classical bilevel scheduling formulations, motivating the development of tailored solution methods that exploit the specific structure of the knapsack-scheduling problem.

3 Solution Methodology

Having formulated the bilevel knapsack-scheduling problem in Chapter 2, we now develop solution methods to find optimal or near-optimal solutions efficiently. We begin by discussing a baseline approach based on complete enumeration

3.1 Complete Enumeration (Baseline)

3.1.1 Algorithm

The most straightforward approach is to enumerate all feasible selections and solve the follower's problem for each:

Algorithm 1 Complete Enumeration for Bilevel Problem

```
C*_max ← 0, x* ← ∅
for each feasible selection x satisfying budget constraint do
    Solve follower's scheduling problem to get C_max(x)
    if C_max(x) > C*_max then
        C*_max ← C_max(x)
        x* ← x
    end if
end for
return (x*, C*_max)
```

3.1.2 Combinatorial Growth

The search space grows exponentially with the problem parameters. Here, the total number of feasible selections is determined by the number of ways to choose non-negative integer quantities x_i for each item type i such that the total cost does not exceed the budget B. Thus, the maximum number of copies that can be purchased for item i is given by

$$\left\lfloor \frac{B}{p_i} \right\rfloor .$$

Because we have to account for the already spent budget from the first d items, its not quite trivial to compute the exact number of feasible combinations. However, an upper bound can be expressed as

$$\prod_{i=1}^{n} \left(\left\lfloor \frac{B}{p_i} \right\rfloor + 1 \right) .$$

For each combination, the algorithm must verify the budget constraint and solve the scheduling problem, which is itself NP-hard.

To illustrate the scale of this challenge, consider an instance with $n = 10$ items, budget $B = 100$, and prices $p_i = 5 + i$ for $i = 0, \ldots, 9$. The maximum number of copies per item is

$(20, 16, 14, 12, 11, 10, 9, 8, 7, 7)$. This yields an upper bound on the number of combinations of 5.29×10^{10}. Assuming an optimistic rate of 1 ms per scheduling problem solve, complete enumeration would require 5.29×10^7 seconds ($\approx$ 612.61 days or 1.68 years), making this approach clearly intractable for practical instances.

3.2 Branch-and-Bound Algorithm

3.2.1 Overview

To overcome the combinatorial explosion of complete enumeration, we develop a branch-and-bound algorithm following the framework of [WF25]. The Branch-and-bound algorithm systematically explores the search space while pruning branches that cannot contain optimal solutions. Here, all possible selections are organized in a search tree, where each node represents a partial selection of items and each leaf node represents a complete selection. The algorithm consists of three main components:

1. **Branching:** We decompose the problem into subproblems
2. **Bounding:** We compute an upper bound on the best possible solution in a subtree
3. **Pruning:** We eliminate subtrees that cannot improve the incumbent

3.2.2 Search Tree Structure

Node representation: Each node represents a partial solution and holds all of the following information:

- **Depth d:** All items $1, \ldots, d$ have fixed quantities.
- **Occurrences:** $x = (x_1, \ldots, x_d, 0, \ldots, 0) \in \mathbb{N}^n$. The first d entries are fixed, the rest are undecided.
- **Remaining budget:** $B_{\text{rem}} = B - \sum_{i=1}^{d} p_i x_i$

Branching rule: From depth d, create children by setting x_{d+1}:

- Branch for $x_{d+1} = 0, 1, 2, \ldots, \lfloor B_{\text{rem}}/p_{d+1} \rfloor$

Example search tree: Figure 3.1 illustrates the branching structure for a problem with 2 items, prices $p_1 = 5$, $p_2 = 8$, and budget $B = 10$.

3.2.3 Follower's Scheduling Algorithm

To evaluate partial solutions in the branch-and-bound tree, we need to compute the makespan for the jobs that have already been selected. For this purpose, we use the **Longest Processing Time (LPT)** rule [Gra69], a well-known greedy algorithm for scheduling jobs on parallel machines (see Algorithm 2 for the procedure used to compute $C_{\max}^{\text{LPT},(d)}$).

Since item types are presorted from longest to shortest duration, the algorithm processes jobs in non-ascending order of processing time, greedily assigning each job to the least loaded machine:

Complexity: $O(J \log m)$ for J jobs (using a priority queue)

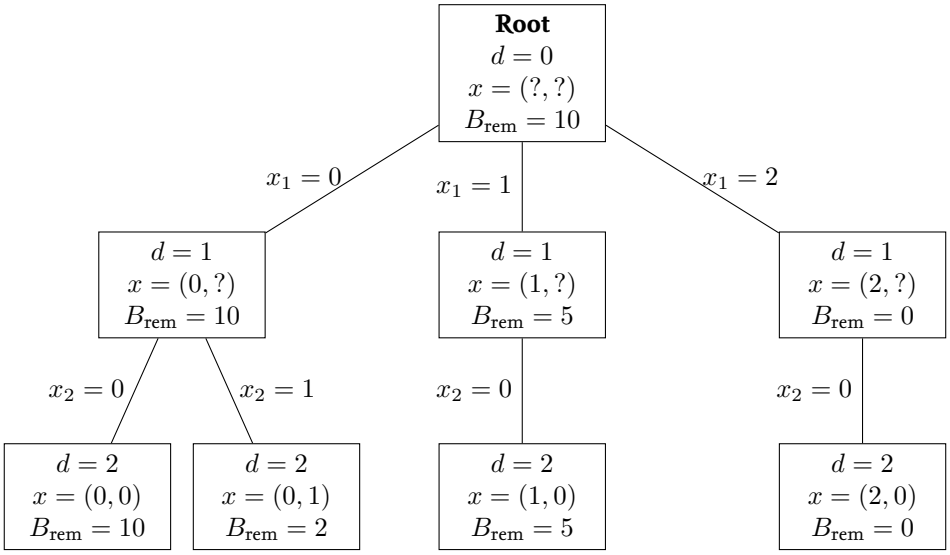

Figure 3.1: Example search tree with 2 items ($p_1 = 5, p_2 = 8, B = 10$). Each node stores depth d, partial solution x, and remaining budget B_{rem}. In this example, item types whose occurrences have yet to be decided are marked with a question mark. Leaf nodes at the bottom represent complete solutions that require solving the follower's problem to determine the makespan.

Algorithm 2 LPT Scheduling for $C_{\max}^{\text{LPT},(d)}$ Computation

```
Initialize machine loads: L_1, ..., L_m ← 0
for each job j with duration d_j (in non-ascending order) do
    k* ← arg min_k L_k                    ▷ Find least loaded machine
    L_{k*} ← L_{k*} + d_j                 ▷ Assign job to machine k*
end for
return C_max^{LPT,(d)} = max_k L_k
```

Optimality: The LPT rule is a $\frac{4}{3}$-approximation for makespan minimization [Gra69], and often produces optimal or near-optimal solutions in practice, as we will see in our computational experiments later.

Incremental Updates: Importantly, we do not recompute $C_{\max}^{\text{LPT},(d)}$ from scratch at each node. Instead, when generating a child node by deciding on item type $d+1$ with quantity x_{d+1}, we incrementally update the machine loads by adding the x_{d+1} jobs of duration d_{d+1} to the current schedule. Similarly, the remaining budget B_{rem} is updated incrementally by subtracting $p_{d+1} \cdot x_{d+1}$ from the parent node's remaining budget. This incremental approach significantly reduces computational overhead during tree traversal.

3.2.4 Upper Bound Computation

Key idea: In the branch-and-bound tree we systematically explore the search space to find the combination of items that maximizes the result of the follower's problem. Given an incumbent solution with makespan $C_{\max}^{\text{inc}}$, we don't want to explore subtrees that cannot yield a better solution.

At a given subtree rooted at depth d, we have already fixed the quantities of item types $1, \ldots, d$. If we can compute an upper bound $\text{UB}(x_1, \ldots, x_d, B_{\text{rem}})$ on the best possible makespan achievable by completing the selection with item types $d+1, \ldots, n$, we can prune the subtree if this upper bound is less than or equal to the incumbent. This follows because no solution in this subtree can improve upon the incumbent and is known in the literature as *bound dominance.*

Formally, since any complete solution satisfies

$$\max_{\substack{x_{d+1},\ldots,x_n \\ \sum_{i=d+1}^{n} p_i x_i \leq B_{\text{rem}}}} C_{\max}^{*}(x_1, \ldots, x_d, x_{d+1}, \ldots, x_n) \leq \text{UB}(x_1, \ldots, x_d, B_{\text{rem}}), \tag{3.1}$$

we can safely prune the subtree whenever

$$\text{UB}(x_1, \ldots, x_d, B_{\text{rem}}) \leq C_{\max}^{\text{inc}}. \tag{3.2}$$

Computing the upper bound: The key challenge is to efficiently compute a tight upper bound $\text{UB}(x_1, \ldots, x_d, B_{\text{rem}})$ that overestimates the best possible makespan in the subtree without solving the full bilevel problem. We achieve this by decomposing the bound into two components: the contribution from already-fixed items $1, \ldots, d$, and an estimate of the contribution from the remaining items $d+1, \ldots, n$.

For the fixed items, we schedule the jobs of types $1, \ldots, d$ with quantities $x_1, \ldots, x_d$ using the Longest-Processing-Time (LPT) rule [Gra69] (see Algorithm 2) to obtain makespan $C_{\max}^{\text{LPT},(d)}$. For the remaining items, rather than enumerating all possible selections, we formulate two different relaxations that provide an upper bound on their contribution to the makespan. The first is a knapsack scheduling relaxation that optimistically estimates the additional makespan contribution from the remaining items under the budget constraint. In the latter part of this work, we will refer to this as the **ceiling bound** since it uses the ceiling function to overestimate the effect of scheduling batches of items on the makespan.

The second relaxation is a Max-LPT relaxation that directly maximizes the LPT makespan contribution from the remaining items, which we will call the **Max-LPT bound** going forward. Both approaches yield valid upper bounds, and we will compare their computational trade-offs in Chapter 5. Both bounds will be discussed in detail in Section 3.3 and Section 3.4, along with efficient algorithms for solving the bounding subproblems.

3.2.5 Branch-and-Bound Algorithm

With the search tree structure, valid upper bounds, and pruning conditions established, we can now present the complete branch-and-bound algorithm. Algorithm 3 integrates all components: it systematically explores the search tree, computes upper bounds at each node, and prunes branches when the bound indicates that no improvement over the incumbent is possible.

A key feature of our algorithm is an **early optimality detection** mechanism we will discuss in Section 3.4.3, which can identify certain instances where the optimal solution can be found at the root node: before entering the main search loop, we solve the Max-LPT problem (see Definition 3.4.1) and verify whether the LPT schedule coincides with the optimal schedule for this solution. If $C_{\max}^{\text{LPT}}(x_{\text{LPT}}) = C_{\max}^{\text{OPT}}(x_{\text{LPT}})$, we have proven that this solution is optimal for the full knapsack-scheduling problem and can return immediately without any branch-and-bound search. Otherwise, we use the Max-LPT solution as a strong initial incumbent and proceed with the search. The algorithm maintains a queue of unexplored nodes and continues until all promising branches have been exhausted.

Algorithm 3 Branch-and-Bound for Bilevel Problem

```
Solve Max-LPT problem to get x_LPT
Compute C_max^LPT(x_LPT) and C_max^OPT(x_LPT)
if C_max^LPT(x_LPT) = C_max^OPT(x_LPT) then
    return (x_LPT, C_max^OPT(x_LPT))                    ▷ Early optimality detection
end if
Initialize: (x*, C*_max) ← (x_LPT, C_max^OPT(x_LPT)), queue Q ← {root node}
while Q ≠ ∅ do
    x ← Q.pop()                                         ▷ Node with partial solution at depth d
    if x is complete solution then
        Solve follower's problem: C_max(x)
        if C_max(x) > C*_max then
            C*_max ← C_max(x), x* ← x                   ▷ Update incumbent
        end if
    else
        Compute B_rem ← B − Σ_{i=1}^{d} p_i x_i         ▷ Remaining budget
        Compute upper bound: UB(x_1, ..., x_d, B_rem)
        if UB(x_1, ..., x_d, B_rem) ≤ C*_max then
            continue                                    ▷ Prune: bound dominated
        end if
        Generate children by branching on next item
        Add feasible children to Q
    end if
end while
return (x*, C*_max)
```

Comment: $C_{\max}^{\text{LPT}}(x_{\text{LPT}})$ denotes the makespan obtained by scheduling the job selection x_{LPT} using the Longest Processing Time (LPT) rule. $C_{\max}^{\text{OPT}}(x_{\text{LPT}})$ denotes the optimal makespan for the same job selection, i.e., the minimum possible makespan achievable by any schedule. Comparing these two values allows early detection of optimality if the LPT rule happens to yield an optimal schedule for x_{LPT}.

3.3 The Ceiling Bound

Having established the branch-and-bound framework, we now develop the two upper bounding methods in detail. We begin with the knapsack scheduling relaxation, which provides an optimistic estimate of the makespan contribution from items not yet selected. We then present the Max-LPT approach, which maximizes the LPT makespan directly and provides provably good approximation guarantees.

3.3.1 Knapsack Scheduling Relaxation

The knapsack scheduling relaxation decomposes the upper bound into the makespan from already-decided items plus an optimistic contribution from the remaining items:

Formulation:

$$\begin{aligned} \text{UB}(x_1, \ldots, x_d, B_{\text{rem}}) = & C_{\max}^{\text{LPT},(d)} + \max_{y_{d+1},\ldots,y_n} \sum_{i=d+1}^{n} d_i \left\lceil \frac{y_i}{m} \right\rceil \\ \text{s.t.} \quad & \sum_{i=d+1}^{n} p_i y_i \leq B_{\text{rem}} \\ & y_i \in \mathbb{Z}_+ \quad \forall i > d \end{aligned} \tag{3.3}$$

where $C_{\max}^{\text{LPT},(d)}$ denotes the makespan obtained by scheduling jobs of types $1, \ldots, d$ with quantities $x_1, \ldots, x_d$ using the LPT rule, and y_i represents the (to-be-determined) quantity of job type i for items not yet fixed.

The objective function uses the ceiling term $\lceil y_i/m \rceil$ to provide an optimistic estimate over the effect of the item types that still need to be selected.

We now establish that this formulation indeed provides a valid upper bound on the optimal follower makespan for any completion of the partial solution.

Theorem 3.3.1 (Upper Bound Validity)**.** The upper bound computed by equation (3.3) is valid, i.e., for any completion of the partial solution $(x_1, \ldots, x_d)$ with items $(x_{d+1}, \ldots, x_n)$, the optimal follower makespan satisfies:

$$C_{\max}^{*}(x_1, \ldots, x_n) \leq \text{UB}(x_1, \ldots, x_d, B_{\text{rem}}). \tag{3.4}$$

Proof. We establish the upper bound by analyzing the Longest-Processing-Time (LPT) scheduling rule. Since LPT is a $\frac{4}{3}$-approximation for the $P||C_{\max}$ problem [Gra69], any upper bound on the LPT makespan provides an upper bound on the optimal makespan.

The Bound is derived as an execution of the LPT rule for the job types $1, \ldots, d$ with quantities $x_1, \ldots, x_d$ and an upper bound on the contribution of the remaining job types $d+1, \ldots, n$ with quantities $y_{d+1}, \ldots, y_n$ chosen to maximize the LPT makespan under the budget constraint.

Consider LPT scheduling on m parallel machines. By construction of the branch-and-bound tree, assume job types are ordered by non-increasing processing time: $d_1 \geq d_2 \geq \cdots \geq d_n$. The LPT rule assigns each job to the currently least-loaded machine.

At depth d in the search tree, quantities $x_1, \ldots, x_d$ are fixed, and all copies of these job types have been scheduled using LPT. Let $C_{\max}^{\text{LPT},(d)}$ denote the makespan after scheduling these jobs. To obtain an upper bound, we assume all machines are filled to exactly $C_{\max}^{\text{LPT},(d)}$—this represents a worst-case scenario that can only increase (never decrease) the final makespan.

Now consider scheduling the remaining job types $d+1, \ldots, n$. When we schedule x_{d+1} copies of type $d+1$ (each with duration d_{d+1}), the first copy increases the makespan by d_{d+1}.

Since all machines have equal load, the makespan increases by d_{d+1} again only after placing copies on all m machines. Therefore, scheduling x_{d+1} copies increases the makespan by

$$\left\lceil \frac{x_{d+1}}{m} \right\rceil \cdot d_{d+1}. \tag{3.5}$$

We apply the same reasoning to each subsequent job type $d+2, \ldots, n$. After scheduling all copies of a job type, we again assume all machines are filled to the current makespan—this assumption only provides a further upper bound and avoids case distinctions.

Thus, the LPT makespan satisfies:

$$C_{\max}^{\text{LPT}} \leq C_{\max}^{\text{LPT},(d)} + \sum_{i=d+1}^{n} \left\lceil \frac{x_i}{m} \right\rceil d_i. \tag{3.6}$$

Now, if we choose variables $x_{d+1}, \ldots, x_n$ to maximize this expression subject to the budget constraint $\sum_{i=d+1}^{n} p_i x_i \leq B_{\text{rem}}$, we obtain the bound in equation (3.3). Since LPT provides a $\frac{4}{3}$-approximation and our bound applies to LPT scheduling, it also bounds the optimal follower makespan. □

The bounding computation requires solving equation (3.3), which is a knapsack problem with a modified objective function. Since the original bilevel knapsack-scheduling problem is already weakly NP-hard, and the bounding subproblem inherits similar computational complexity, solving it at every node of the branch-and-bound tree would be prohibitively expensive. To overcome this bottleneck, we adopt a preprocessing strategy: we solve the bounding subproblem once before the branch-and-bound algorithm starts and store all relevant solutions in a table. During the branch-and-bound traversal, we simply look up precomputed entries from this table in constant time, avoiding redundant computations and significantly accelerating the overall algorithm.

3.3.2 Dynamic Programming Approach

We use dynamic programming to solve the ceiling knapsack problem efficiently:

State: $f[i][b]$ = maximum value using items $1, \ldots, i$ with budget b

Recurrence:

$$f[i][b] = \max_{k=0}^{\lfloor b/p_i \rfloor} \left\{ f[i-1][b - k \cdot p_i] + d_i \left\lceil \frac{k}{m} \right\rceil \right\} \tag{3.7}$$

Complexity: $O(nBK)$ where $K = \max_i \lfloor B/p_i \rfloor$

Practical Implementation: In practice, we can use a standard knapsack solver that maximizes the sum of durations. For each item type i, we create:

- One copy of item i with duration d_i and cost p_i (representing taking 1 to $m-1$ copies)
- Multiple "packages of item type i" with cost $m \cdot p_i$ and duration d_i each (representing batches of m items)

This transformation allows us to use efficient off-the-shelf knapsack solvers while correctly capturing the ceiling effect in the objective function.

3.3.3 Optimization: Reversed Item Order

To efficiently compute bounds at different depths, we pre-solve the DP table with items in reverse order. This allows us to query "last k items" efficiently.

Example: Consider a node at depth $d = 2$ where we have decided on items x_1 and x_2, with remaining budget $B_{\text{rem}} = 50$. To compute the upper bound, we need to solve the knapsack scheduling relaxation over items $\{3, 4, \ldots, n\}$ with budget B_{rem}. Since we preprocessed the DP table with items in reverse order (starting from item n down to item 1), we can directly look up the entry corresponding to "items $\{3, \ldots, n\}$ with budget 50" in constant time. Specifically, we query $f[n-2][50]$, which gives us the maximum achievable makespan contribution from the remaining items, thus providing the second term in equation (3.3).

3.4 Another Upper Bound: Maximizing the LPT Rule

While the branch-and-bound algorithm uses the LPT rule to model the follower's scheduling behavior, we can exploit this connection to derive tighter bounds and better initial solutions. Since maximizing the LPT makespan provides a provably good approximation to the leader's optimal solution, we can use this approach in two ways: first, solving the Max-LPT problem before starting the branch-and-bound algorithm yields a strong initial incumbent; second, applying the Max-LPT approach at each node in the search tree provides an alternative upper bound for pruning. We will compare the computational trade-offs between different bounding strategies experimentally in Chapter 5.

3.4.1 The Max-LPT Problem

Recall that the **Longest Processing Time (LPT) rule** [Gra69] schedules jobs in non-increasing order of their processing times, always assigning the next job to the machine with the smallest current load. This greedy rule is well-known to provide good makespan approximations for parallel machine scheduling. We will confirm these observations in our computational experiments in Chapter 5, where we find that the LPT rule often yields optimal or near-optimal solutions for the follower's scheduling problem.

We now define the **Max-LPT problem** as follows:

Definition 3.4.1 (Max-LPT Problem)**.** Given n item types indexed by $i = 1, \ldots, n$, where each item type i has duration d_i and cost p_i, a number of machines m, and a budget constraint B, the Max-LPT problem seeks to find a combination of items $(x_1, \ldots, x_n)$ with $x_i \geq 0$ for all i (allowing multiple copies of each item type) such that:

- The budget constraint is satisfied: $\sum_{i=1}^{n} p_i x_i \leq B$
- The makespan is maximized when the selected jobs are distributed on the machines using the LPT rule

In other words, the Max-LPT problem asks: what is the best job selection that maximizes the makespan achievable by the LPT scheduling rule?

3.4.2 Approximation Guarantee

Before presenting the algorithm for solving the Max-LPT problem, we establish its connection to the original knapsack-scheduling problem.

Theorem 3.4.1 (3/4-Approximation Guarantee)**.** Let $C_{\max}^{\text{OPT}}(x)$ denote the optimal value of the scheduling problem for a job selection x. Then

$$\frac{3}{4} \cdot C_{\max}^{\text{OPT}}(x^*) \leq C_{\max}^{\text{OPT}}(\hat{x}) \leq C_{\max}^{\text{OPT}}(x^*).$$

where $\hat{x}$ is an optimal solution to the Max-LPT problem and x^* is an optimal solution to the original knapsack-scheduling problem.

To prove this result, we first establish two general results about approximation algorithms and their application to bilevel optimization problems.

Lemma 3.4.2 (Properties of approximation algorithms)**.** Let `approx` be an approximation algorithm for a minimization problem with approximation constant $\alpha \geq 1$. Then there exists a constant $\beta := \frac{1}{\alpha}$ such that

$$\beta \cdot \mathrm{Opt} \leq \beta \cdot \mathrm{approx} \leq \mathrm{Opt} \leq \mathrm{approx}.$$

Proof. Since `approx` is an approximation algorithm for a minimization problem with approximation constant α, it holds that

$$\mathrm{Opt} \leq \mathrm{approx} \leq \alpha \cdot \mathrm{Opt}.$$

Setting $\beta = \frac{1}{\alpha}$ and multiplying both sides by β, we obtain

$$\beta \cdot \mathrm{Opt} \leq \beta \cdot \mathrm{approx} \leq \mathrm{Opt} \leq \mathrm{approx}.$$

which proves the claim. □

Theorem 3.4.3 (Approximation algorithms and bilevel optimization problems [WF25])**.** Let a bilevel optimization problem $\max_{x \in \mathcal{X}} \min_{y \in \mathcal{L}(x)} c(x, y)$ be given and let approx be a function such that there exists a constant $\beta \leq 1$ fulfilling:

1. $\mathrm{approx}(x) \in \mathcal{L}(x)$ for all $x \in \mathcal{X}$,
2. $\beta \cdot c(x, y^*(x)) \leq \beta \cdot c(x, \mathrm{approx}(x)) \leq c(x, y^*(x)) \leq c(x, \mathrm{approx}(x))$

where $y^*(x)$ denotes an optimal lower-level solution for fixed x.

If $\hat{x} \in \arg\max_{x \in \mathcal{X}} c(x, \mathrm{approx}(x))$, then it follows that

$$\beta \cdot \mathrm{Opt} \leq c(\hat{x}, y^*(\hat{x})) \leq \mathrm{Opt},$$

i.e., $c(\hat{x}, y^*(\hat{x}))$ is a β-approximate solution of the bilevel problem $\max_{x \in \mathcal{X}} \min_{y \in \mathcal{L}(x)} c(x, y)$.

Proof. By definition of $\hat{x}$ we have

$$c(x^*, \mathrm{approx}(x^*)) \;\leq\; c(\hat{x}, \mathrm{approx}(\hat{x})),$$

where x^* is an optimal upper-level solution.

Putting everything together, we obtain

$$\beta \cdot \mathrm{Opt} = \beta \cdot c(x^*, y^*(x^*)) \;\leq\; \beta \cdot c(x^*, \mathrm{approx}(x^*)) \;\leq\; \beta \cdot c(\hat{x}, \mathrm{approx}(\hat{x})) \;\leq\; c(\hat{x}, y^*(\hat{x})).$$

This proves the claim. □

Remark: The proof above follows the approach of [WF25], who proved an analogous result for bilevel min-max problems. While their setting considers minimization in the upper level and maximization in the lower level, their arguments are basically identical.

Proof of Theorem 3.4.1. We apply Theorem 3.4.3 to our knapsack-scheduling problem. The LPT rule is known to be a $\frac{4}{3}$-approximation algorithm for the scheduling problem on parallel machines. By Lemma 3.4.2, this means that for any job selection x and $\beta = \frac{3}{4}$, we get:

$$\frac{3}{4} \cdot \mathrm{Opt}(x) \leq \frac{3}{4} \cdot C_{\max}^{\mathrm{LPT}}(x) \leq \mathrm{Opt}(x) \leq C_{\max}^{\mathrm{LPT}}(x)$$

where $\mathrm{Opt}(x)$ is the optimal makespan for the job selection x and $C_{\max}^{\mathrm{LPT}}(x)$ is the makespan obtained by applying the LPT rule to the same job selection. Since the schedules generated by the LPT rule are feasible for all job selections $x \in \mathcal{X}$, we fulfill the conditions of Theorem 3.4.3.

Now, let $\hat{x}$ be the solution that maximizes the LPT makespan, i.e., $\hat{x}$ solves the Max-LPT problem and let x^* be the optimal job selection for the knapsack-scheduling problem. Applying Theorem 3.4.3 with $\beta = \frac{3}{4}$, we obtain:

$$\frac{3}{4} \cdot C_{\max}^{\mathrm{OPT}}(x^*) \leq C_{\max}^{\mathrm{OPT}}(\hat{x}) \leq C_{\max}^{\mathrm{OPT}}(x^*).$$

This proves that the solution obtained by maximizing the LPT makespan and then solving the corresponding scheduling problem is a $\frac{3}{4}$-approximation to the optimal solution of the knapsack-scheduling problem. □

This result is significant because it guarantees that solving the Max-LPT problem provides a constant-factor approximation for the harder problem of finding the optimal job selection under optimal scheduling. Moreover, as we will show later, the Max-LPT problem can be solved optimally in pseudo-polynomial time.

3.4.3 Early Optimality Detection via Max-LPT

While the Max-LPT approach provides a $\frac{3}{4}$-approximation guarantee in the worst case, it can sometimes find the exact optimal solution to the knapsack-scheduling problem without requiring the full branch-and-bound search. This observation has significant practical implications. Recall that for any job selection x, the LPT makespan provides an upper bound on the optimal makespan:

$$C_{\max}^{\mathrm{LPT}}(x) \geq C_{\max}^{\mathrm{OPT}}(x).$$

Now suppose x^* is an optimal solution to the Max-LPT problem, i.e., $x^* \in \arg\max_{c^T x \leq B} C_{\max}^{\mathrm{LPT}}(x)$. If we have that $C_{\max}^{\mathrm{LPT}}(x^*) = C_{\max}^{\mathrm{OPT}}(x^*)$, then for any other feasible job selection x we have:

$$C_{\max}^{\mathrm{OPT}}(x^*) = C_{\max}^{\mathrm{LPT}}(x^*) \geq C_{\max}^{\mathrm{LPT}}(x) \geq C_{\max}^{\mathrm{OPT}}(x).$$

This chain of inequalities shows that if this property holds, x^* is not only optimal for the Max-LPT problem but also for the full knapsack-scheduling problem with optimal scheduling.

Practical significance: Since we compute $C_{\max}^{\mathrm{OPT}}(x^*)$ anyway to establish the initial incumbent for the branch-and-bound algorithm, we can verify whether $C_{\max}^{\mathrm{LPT}}(x^*) = C_{\max}^{\mathrm{OPT}}(x^*)$ holds at no additional cost. When this equality holds, we can skip the entire search and solve even complex problems within seconds. As we will demonstrate in Chapter 5, this situation occurs in roughly 84.4% of instances, substantially reducing the computational effort required. This may be seen as the main contribution of this work.

3.4.4 Ordered Makespan Notation

Before presenting the algorithm, we introduce notation for the ordered machine loads in a schedule. Given a schedule that assigns jobs to m machines, let $L_1, L_2, \ldots, L_m$ denote the total processing times (loads) on each machine. We define the **ordered makespan sequence** as follows:

Definition 3.4.2 (Ordered Makespan)**.** For a given schedule on m machines, let $L_1, L_2, \ldots, L_m$ denote the machine loads. Sort these loads in non-increasing order as $L_{\sigma(1)} \geq L_{\sigma(2)} \geq \cdots \geq L_{\sigma(m)}$ for some permutation σ. We define:

- $C_{\max}^{(1)}$ is the **standard makespan**, i.e., the maximum load: $C_{\max}^{(1)} = L_{\sigma(1)} = \max_{k=1,\ldots,m} L_k$
- $C_{\max}^{(2)}$ is the **second-largest** machine load: $C_{\max}^{(2)} = L_{\sigma(2)}$
- More generally, $C_{\max}^{(i)} = L_{\sigma(i)}$ is the i-th largest machine load for $i = 1, 2, \ldots, m$
- $C_{\max}^{(m)}$ is the **minimum** machine load: $C_{\max}^{(m)} = L_{\sigma(m)} = \min_{k=1,\ldots,m} L_k$

Note on ties: When multiple machines achieve the same load, they occupy consecutive positions in the sorted order. For instance, if two machines both achieve the maximum load $C_{\max}^{(1)}$, then $C_{\max}^{(2)} = C_{\max}^{(1)}$. This notation allows us to refer to machines by their relative load ordering rather than their indices.

3.4.5 Algorithm Description and Correctness

The key observation is that in an optimal solution, the last job i that was scheduled either changed the makespan or it did not. If it did, we know that the last job i was put on the machine corresponding to $C_{\max}^{(m)}$ and since we have an optimal solution, there cannot be a combination of jobs so that the LPT rule would deliver a larger $C_{\max}^{(m)}$ given the budget $B - p_i$. If the last job that was scheduled did not change the makespan, we know that there has to be another job j that was scheduled earlier and the combination of jobs before the job j maximizes the $C_{\max}^{(m)}$ at the time j was placed. All jobs that are scheduled after j do not change the makespan by our assumption. We use this observation to solve the max-LPT problem: By considering all item types as candidates for the last job and optimizing the preceding schedule accordingly, we can find the optimal solution to the max-LPT problem.

Theorem 3.4.4. The Algorithm 4 MAX-LPT always computes an optimal solution for the MAX-LPT problem.

Algorithm 4 MAX-LPT

Require: Budget B, item types $1, \ldots, n$ with durations d_i and costs p_i for each item $i \in \{1, \ldots, n\}$, number of machines m
Ensure: Optimal value for the Max-LPT problem

```
Precompute the DP table DP for the unbounded knapsack problem
Initialize incumbent z* ← 0
for each item J ∈ {1, ..., n} do
    Compute B* := ⌊B/m⌋
    B̂ := B − m · B*
    if B̂ < p_J then
        k ← 0
        while B̂ < p_J and B* > 0 do
            k ← k + 1
            B* := ⌊(B − k·m)/m⌋
            B̂ := B − m · B*
        end while
        if B̂ < p_J then
            continue                    ▷ Skip this item if budget still insufficient
        end if
    end if
    z ← DP(J, B*) + d_J
    if z > z* then
        z* ← z
    end if
end for
return z*
```

Proof. We begin with the key observation:

In an optimal solution of the max-LPT problem, the job I that finishes last is assigned to the machine with the shortest total processing time $C_{\max}^{(m)}$ at that moment. Therefore, the makespan is determined by the duration of this final job d_I together with $C_{\max}^{(m)}$ at that moment. Since we assume to have an optimal solution, there cannot be a combination of jobs so that the LPT rule would deliver a larger $C_{\max}^{(m)}$ for the unbounded knapsack subproblem that maximizes $C_{\max}^{(m)}$ with budget $B - p_I$ and job types $1, \ldots, I$ available. Otherwise, we could replace the combination of jobs in the optimal solution before putting I on the least loaded machine with the combination that maximizes $C_{\max}^{(m)}$ and then place item I on the least loaded machine, which contradicts the optimality of the solution. We exploit this property by considering each item type $J \in \{1, \ldots, n\}$ as a candidate for the job that finishes last. For a fixed candidate J, we maximize the $C_{\max}^{(m)}$ for the remaining jobs under the remaining budget.

We now note some observations about the structure of optimization problems looking to maximize $C_{\max}^{(m)}$ under a budget constraint: Considering any schedule, we can define the **per-machine-budget** of a machine M as the sum of the prices of all jobs assigned to this machine. If now the following holds for a combination of items and the resulting schedule:

- **Property 1:** The per-machine-budget B^* of the least loaded machine i is highest-possible. This means that for all machines, we cannot redistribute the budget to achieve a higher per-machine-budget for the least loaded machine.

- **Property 2:** with this highest possible budget B^*, the best possible combination of job types from among $1, \ldots, J$ was acquired for the least loaded machine,

then there cannot be a combination of jobs that would yield a larger $C_{\max}^{(m)}$ for the budget available and job types $1, \ldots, J$ available.

By using the LPT rule and a dynamic programming table, we can force a schedule that fulfills the above properties. Let

$$B^* := \left\lfloor \frac{B}{m} \right\rfloor,$$

which is exactly the highest possible per-machine budget for the machine that gets the least amount of budget. Let

$$\hat{B} = B - m \cdot B^*$$

be the remaining budget.

If $\hat{B} \geq p_J$, the final job of type J can be paid without reducing the per-machine budget. This corresponds to the case where no adjustment is needed in the algorithm. By evaluating the unbounded knapsack solution stored in $DP(J, B^*)$, we can reconstruct the best possible combination of item types so that the sum of durations is maximized with budget B^* and item types $1, \ldots, J$ available. If we buy m copies of every item in this combination and place them according to the LPT rule, we will assign the combination of items yielded by $DP(J, B^*)$ to each of the m machines. Since by following this procedure, we will fulfill **Property 1** and **Property 2** mentioned above, there cannot be a combination of jobs that would yield a larger $C_{\max}^{(m)}$ for the budget $m \cdot B^*$ and job types $1, \ldots, J$ available. Adding the duration of the final job d_J gives us the total makespan for this candidate solution.

If $\hat{B} < p_J$, the per-machine budget B^* must be reduced uniformly until the remaining budget is large enough to pay for item J. After this adjustment, the optimal combination of jobs scheduled on the least loaded machine before adding J is once again obtained from the dynamic programming table $DP(J, B^*)$. This corresponds to the adjustment loop in the algorithm.

Since the algorithm evaluates all item types as possible final jobs and, for each of them, computes the optimal preceding schedule, it considers all candidates for an optimal solution. By selecting the maximum makespan among these candidates, the algorithm returns an optimal solution to the max-LPT problem. □

Running time. The running time of the algorithm is

$$\mathcal{O}(n \cdot B).$$

This is because we compute the DP table for the unbounded knapsack problem once, which takes $\mathcal{O}(n \cdot B)$ time, and then we iterate through each of the n item types as candidates for the last job. For each candidate, we perform a constant number of operations (looking up the DP table and adjusting the budget no more than B times if necessary), resulting in an overall running time of $\mathcal{O}(n \cdot B)$.

3.4.6 Connection to the Knapsack-Scheduling Problem

The max-LPT algorithm provides a good approximation to the leader's problem by maximizing the makespan under the LPT scheduling rule. While the LPT rule is itself an approximation for optimal scheduling, this algorithm finds the *optimal* job selection that maximizes the LPT makespan. We have already shown that this approach yields a $\frac{3}{4}$-approximation for the original knapsack-scheduling problem (Section 2). If we solve the max-LPT problem

before calling the branch and bound Algorithm of the bilevel knapsack program, we obtain a starting incumbent that is guaranteed to be within $\frac{3}{4}$ of the optimal solution. In the branch and bound algorithm itself, we can use the max-LPT algorithm to compute upper bounds at each node by evaluating the max-LPT solution for the remaining budget and item types. This provides an upper bound with a $\frac{3}{4}$-approximation guarantee. The disadvantage of this approach is that we have to compute the unbounded knapsack DP table for each depth d in the branch and bound tree, since the remaining item types change at each depth: In the max-LPT algorithm, we need to solve the unbounded knapsack problem for the remaining item types $d+1, \ldots, n$ at depth d. Since we have to regard every item type $i \in \{d+1, \ldots, n\}$ as a candidate for the last job that finishes, we need to be able to evaluate the best possible combination of item types $d+1, \ldots, i$ for each budget. The unbounded knapsack DP tables computed before start from item type 1 up to item type d and therefore cannot be used again as we increase the depth in the branch and bound tree. In total, we have to compute n DP tables, one for each possible depth in the tree. Each DP table requires O((n-d)B) time to compute. This is computationally more expensive than our previous upper bound, which only required a single DP table computed once before the branch and bound traversal. We compare both upper bounds experimentally in Chapter 5 to evaluate the trade-off between tighter bounds and computational overhead.

Example: Computing the Max-LPT Upper Bound at Different Depths

To illustrate how the Max-LPT upper bound is computed at different depths in the branch-and-bound tree, consider the following example.

Assume $n = 4$ item types, budget $B = 20$, $m = 2$ machines, and the item types are ordered in non-increasing order of duration:

Item type i	1	2	3	4
Duration d_i	8	6	4	2
Cost p_i	7	5	4	2

Suppose we are at two different depths d in the branch-and-bound tree:

Depth $d = 1$ (only x_1 fixed)

x_1 is fixed, while x_2, x_3, x_4 are undecided.

Remaining budget:

$$B_{\text{rem}} = B - p_1 x_1$$

To compute the Max-LPT upper bound, we solve the Max-LPT problem for the remaining item types $2, 3, 4$ and budget B_{rem}.

This requires a DP table for the unbounded knapsack problem with items $2, 3, 4$ (i.e., the longest available duration is d_2).

For each possible candidate for the last job (item 2, 3, or 4), we maximize the per-machine budget and use the DP table to find the best combination of jobs for the remaining budget.

This means we need to be able evaluate the DP table for item type 2 available, item types 2 and 3 available, and item types $2, 3$ and 4 available to compute the upper bound at this node.

Depth $d = 2$ (both x_1 and x_2 fixed)

x_1, x_2 are fixed, while x_3, x_4 are undecided.

Remaining budget:

$$B_{\text{rem}} = B - p_1 x_1 - p_2 x_2$$

We solve the Max-LPT problem for the remaining item types $3, 4$ and budget B_{rem}.

This requires a DP table for the unbounded knapsack problem with items $3, 4$ (i.e., the longest available duration is d_3).

For each possible candidate for the last job (item 3 or 4), we again maximize the per-machine budget and use the DP table to find the best combination.

This means we need to be able evaluate the DP table for item type 3 available and item types 3 and 4 available to compute the upper bound at this node.

Since the DP table computed for depth $d = 1$ only includes items $2, 3, 4$, it cannot be reused at depth $d = 2$ where only items $3, 4$ are available.

DP Table Requirements and Item Ordering

In general, at each depth d, we need a DP table for the unbounded knapsack problem with items $d + 1, \ldots, n$. Since the items are ordered by non-increasing duration, this means:

- At depth $d = 0$ (root): DP table for items $1, 2, 3, 4$ (all items available).
- At depth $d = 1$: DP table for items $2, 3, 4$ (longest is d_2).
- At depth $d = 2$: DP table for items $3, 4$ (longest is d_3).
- At depth $d = 3$: DP table for item 4 only.

Thus, to efficiently compute the Max-LPT upper bound at any node, we must be able to read the DP table for the case when the shortest item type still available is d_{d+1}, then d_{d+2}, and so on, depending on the candidate for the last item type.

This example demonstrates why n different DP tables are needed for the Max-LPT upper bound approach—one for each possible set of remaining item types as we progress through the tree.

3.4.1 Modified Branch-and-Bound with Max-LPT Upper Bound

To use the Max-LPT approach as an alternative upper bound in the branch-and-bound algorithm, we modify the bounding step as shown in Algorithm 5. The key difference from Algorithm 3 is that instead of computing the upper bound using the ceiling knapsack relaxation (equation (3.3)), we call the Max-LPT algorithm on the remaining item types with the remaining budget. This requires precomputing a separate DP table for each depth level, as the set of remaining items changes as we descend the tree. Like Algorithm 3, this variant also incorporates the early optimality detection 3.4.3 mechanism to potentially avoid the search entirely.

Algorithm 5 Branch-and-Bound with Max-LPT Upper Bound

```
Precompute DP tables DP_d for each depth d = 1, ..., n
Solve Max-LPT problem to get x_LPT
Compute C_max^LPT(x_LPT) and C_max^OPT(x_LPT)
if C_max^LPT(x_LPT) = C_max^OPT(x_LPT) then
    return (x_LPT, C_max^OPT(x_LPT))                    ▷ Early optimality detection
end if
Initialize: (x*, C*_max) ← (x_LPT, C_max^OPT(x_LPT)), queue Q ← {root node}
while Q ≠ ∅ do
    x ← Q.pop()                                         ▷ Node with partial solution at depth d
    if x is complete solution then
        Solve follower's problem: C_max(x)
        if C_max(x) > C*_max then
            C*_max ← C_max(x), x* ← x                   ▷ Update incumbent
        end if
    else
        Compute B_rem ← B − Σ_{i=1}^{d} p_i x_i         ▷ Remaining budget
        Compute C_max^{LPT,(d)} ← LPT makespan for items 1, ..., d
        Compute UB_Max-LPT ← C_max^{LPT,(d)} + Max-LPT(d + 1, ..., n, B_rem, DP_d)
        if UB_Max-LPT ≤ C*_max then
            continue                                    ▷ Prune: bound dominated
        end if
        Generate children by branching on next item
        Add feasible children to Q
    end if
end while
return (x*, C*_max)
```

4 Implementation Details

4.1 Software Architecture

The implementation is modular, with key components for the branch-and-bound search, problem modeling, and scheduling:

- **models.py:** Data structures for problem instances and search nodes
- **bnb.py:** Branch-and-bound solver with pruning and search tree logic
- **solvers.py:** Scheduling subproblems solved via Gurobi [Gur26], version 12.0.3.
- **bilevel_gurobi.py:** Complete enumeration baseline for correctness verification
- **test_big.py:** Contains the PatternTestSet with seven distinct patterns for instance generation, used in computational experiments.
- **test_grid_sensitivity.py:** Contains the GridTestSet for sensitivity analysis on three different budget multipliers and varies machine counts and number of job types independently.

4.2 Key Data Structures

4.2.1 Problem Node

Each node in the branch-and-bound search tree represents a partial or complete leader decision. The node tracks job selections, decision depth, remaining budget, and machine loads for efficient makespan evaluation. The implementation supports efficient branching and pruning:

```
class ProblemNode:
    job_occurrences: List[int]     # Number of jobs selected for each job type
    depth: int                     # Current decision depth (job type index)
    remaining_budget: int          # Budget left after current selections
    n_job_types: int               # Total number of job types
    _machine_loads: List[float]    # Sorted list of loads on each machine (internal)
    _branch_type: str              # 'root', 'right', 'left' (branching type)
    _m: int                        # Number of machines
```

Branch types:

For efficient branching and pruning, each node is classified into one of three types based on how it was generated from its parent:

- `'root'`: This node type increased the depth. Can create both left and right children; jobs at current depth are not yet distributed to machines.
- `'right'`: This node type was created by incrementing the current job type by one; Can create right and lower children; jobs at depth are distributed incrementally to machines by the LPT rule to track makespan contributions.
- `'left'`: This node type was created by decrementing the current job type by one; Can create left and lower children; jobs at depth are not distributed, loads are copied unchanged. Jobs at depth are only distributed when evaluating makespan or committing to the next depth level, because when more left children are generated from a left node, the number of job types at the current depth decreases and we would need to recalculate the distribution of jobs to machines.

The makespan contribution from committed jobs is incrementally tracked and updated using the internal machine load state. Each node distributes job occurrences to machines following the LPT rule, ensuring efficient evaluation without recomputation. This approach is computationally efficient and provides good approximate makespan values for partial leader decisions.

4.3 Search Tree Structure and Algorithmic Optimizations

4.3.1 Depth-First Search and Node Expansion

The branch-and-bound algorithm uses a depth-first search strategy:

- The search starts with a provably good incumbent solution, obtained by solving the MaxLPT problem.
- At each node, children are generated corresponding to possible decisions for the next job type.
- Left children correspond to decrementing the quantity of the current job type, while right children increment the quantity. Both directions are explored to cover all feasible allocations within the budget.
- The search prioritizes lower children and makes no distinction between left or right children; all feasible branches are explored according to the branch-and-bound logic, ensuring optimality and efficient pruning.
- At the leaf level, only a single leaf node is generated: the one that spends the most remaining budget and thus acquires the maximum number of jobs of type n. This is optimal, as any other leaf node with fewer jobs cannot yield a greater makespan.
- This approach avoids unnecessary expansion of all possible leaf nodes, improving efficiency, since evaluating leaf nodes by solving the follower's scheduling problem is computationally expensive.
- Memory usage is minimized compared to breadth-first approaches.
- The depth-first strategy quickly evaluates complete solutions, which leads to early updates of the incumbent and more effective pruning of suboptimal branches.

4.3.2 Incumbent Initialization

The initial incumbent is computed by solving the MaxLPT relaxation:

- The MaxLPT problem provides a provable upper bound and a feasible leader solution.
- The resulting job selection is passed to the Gurobi scheduler [Gur26], which solves the follower's scheduling problem optimally via integer programming.
- This guarantees a strong starting incumbent and enables immediate pruning for many instances.

4.3.3 Incremental Budget and Contribution Tracking

Each node tracks remaining budget and incremental makespan contributions:

- Remaining budget is updated in constant time from parent to child.
- The makespan contribution from decided items is cached and updated incrementally, avoiding recomputation.

5 Computational Experiments

This chapter presents a comprehensive computational study evaluating the performance of our branch-and-bound algorithm for the bilevel knapsack-scheduling problem. We systematically test the algorithm's efficiency, scalability, and effectiveness through multiple experimental phases: verification against complete enumeration, sensitivity analysis exploring individual parameter scaling, grid-based experiments examining combined parameter effects, and benchmarking against a state-of-the-art bilevel solver.

5.1 Experimental Setup

5.1.1 Test Environment

All experiments were conducted on a single-threaded configuration with the following specifications:

- **Hardware:** AMD Ryzen 5 9600X 6-Core Processor (12 logical processors), 32 GB RAM
- **Software:** Python 3.11, Gurobi [Gur26] 12.0.3

5.1.2 Performance Metrics

For each test instance, we recorded:

- **Runtime:** Wall-clock time in seconds
- **Nodes explored:** Number of branch-and-bound nodes evaluated
- **Pruning rate:** Percentage of nodes at which we pruned, i.e., did not further expand the search tree
- **Solution quality:** Optimal makespan $C^*_{\max}$
- **Status:** root_optimal (optimal solution found at the root node), completed, timeout (overall time limit exceeded), or leaf timeout (individual leaf node evaluation exceeded time limit)

5.1.3 Test by Pattern

To comprehensively evaluate algorithm behavior across diverse problem structures, we developed `PatternTestSet`, a structured benchmark consisting of 140 instances systematically generated using seven distinct patterns (20 instances per pattern). Each pattern targets specific cost-efficiency characteristics and scheduling challenges:

Pattern 1: Uniform Ratios

- Jobs with consistent duration-to-price ratios (1.5–3.0)

- Durations: 3–15, prices calculated from ratio
- Creates balanced, predictable instances with similar cost-efficiency across all job types
- Tests algorithm performance on well-structured instances

Pattern 2: High Variance

- Alternates between cheap-long jobs (duration 10–20, price 1–3) and expensive-short jobs (duration 2–5, price 8–15)
- High variance in duration-to-price ratios
- Tests algorithm's ability to handle mixed job portfolios with extreme cost-time trade-offs

Pattern 3: Increasing Complexity

- Systematic progression: duration $= 3 + 2j$, price $= 2 + j$ for job type j
- Jobs become progressively longer and more expensive
- Structured, predictable difficulty gradient

Pattern 4: Random Moderate

- Prices: 5–50, durations: 3–30
- Moderate ranges with uniform random distribution
- Balanced between structure and randomness

Pattern 5: Extreme

- Prices: 1–100, durations: 1–50
- Wide ranges test algorithm robustness to outliers
- Potential for highly unbalanced solutions

Pattern 6: Strong Correlation

- Strong positive correlation between price and duration
- Duration $\approx 5 \times$ price with small random perturbations
- Tests algorithm when cost and benefit are closely aligned
- Instance generation inspired by [WF25; Pis05]

Pattern 7: Subset Sum

- Prices form subset sum instance structure
- Durations independent of prices
- Stresses upper-level knapsack decision complexity
- Instance generation inspired by [WF25; Pis05]

Instance sizes range from 6–12 job types, 3–8 machines, and budgets 15–50. Instances are designated Complex_001 through Complex_140.

5.1.4 Verification Study: Algorithm Correctness

To verify correctness and demonstrate computational efficiency, we compared our branch-and-bound results against complete enumeration (the most straightforward and exhaustive approach for problems of this type) on all 140 instances in the PatternTestSet. Enumeration guarantees optimality by evaluating every feasible leader decision, but is prohibitively slow. Even for relatively simple instances, it sometimes exceeds the 1-hour time limit. In contrast, our branch-and-bound algorithm consistently solved the same instances in fractions of seconds, always matching the optimal makespans found by enumeration (when available).

Verification results: Out of 140 instances, branch-and-bound and enumeration produced identical optimal makespans for available cases. Enumeration completed within the time limit for 125 instances (89%), but timed out (runtime $\geq$ 3,600 seconds) on 15 instances (11%). Even for these cases, branch-and-bound still found and verified the optimal solution rapidly. No failures or discrepancies were observed. This confirms both the correctness and the dramatic speed advantage of our method over the baseline enumeration approach.

Table 5.1: Verification and runtime summary by pattern in seconds

Pattern	TO	Mean Enum	Mean Ceil	Mean LPT	Max Ceil	Max LPT
Uniform Ratios	6/20	1194.53	0.0032	0.0041	0.0110	0.0175
High Variance	6/20	1162.13	0.0078	0.0088	0.0682	0.0603
Increasing	0/20	70.98	0.0015	0.0019	0.0060	0.0035
Random Moderate	1/20	422.17	0.0027	0.0031	0.0105	0.0105
Extreme	2/20	856.36	0.0220	0.0185	0.3291	0.2630
Strong Correlation	0/20	45.34	0.0011	0.0018	0.0030	0.0045
Subset Sum	0/20	27.67	0.0009	0.0013	0.0020	0.0022

Note: *TO = number of enumeration timeouts out of 20 instances per pattern.*

Since we could not observe significant time differences between the different creation patterns due to the overall speed of our algorithm, we decided to focus on a sensitivity analysis in the next phase to explore the limits of our approach, where we systematically increase individual parameters to observe how performance scales.

5.2 Sensitivity Analysis: Single Parameter Scaling

Having verified correctness, we investigated how algorithm performance scales as individual problem parameters increase. For each parameter (number of job types, number of machines, budget multiplier), we conducted a systematic sensitivity analysis using a 1-hour time limit per instance. For each parameter value, 5 random instances were generated and solved using both the ceiling and MaxLPT bounds. The parameters were increased until we hit the time limit.

Instance generation: For each parameter setting, test instances were generated using the following procedure:

- Job prices: Uniformly random integers in [10, 100]
- Job durations: Uniformly random integers in [5, 50]
- Budget: $B =$ budget multiplier$\times$ (average job cost) $\times$ (number of machines)

- Baseline: 20 jobs, 10 machines, budget multiplier 2.5, node limit 1 billion, time limit 1 hour per instance, leaf timeout 15 minutes
- Number of machines, jobs, and budget multiplier varied as specified
- Each instance uses a unique random seed for reproducibility

For each instance, the branch-and-bound algorithm was run twice: once with the ceiling bound and once with the MaxLPT bound. To capture the full performance profile, we generated instances until at least one instance used the branch and bound tree instead of being solved at the root node for each parameter setting, allowing us to identify the practical limits of our approach and compare the performance of the two bounds under increasing complexity. Performance metrics were recorded for all runs to analyze trends and differences between the bounds as parameters scaled up.

5.2.1 Methodology

- Fix two parameters at baseline values
- Systematically increase the third parameter
- For each parameter value, generate instances with different random seeds until at least one instance requires branch-and-bound search (not solved at root)
- Continue increasing the observed parameter until at least one instance times out (exceeds 3,600 seconds)
- Record runtime, nodes explored, pruning rate, and solution quality

Column meanings and discussion. The sensitivity tables report performance metrics as a single parameter (number of job types, machines, or budget multiplier) increases. Columns include average runtime in seconds for the ceiling- and MaxLPT bound, the average number of branch-and-bound nodes explored, the average pruning rate (percentage of nodes at which we pruned), and the number of instances that were generated until the problem was not solved at the root node due to the early optimality detection mechanism described in Section 3.4.3 (since this mechanism works identically for both bounds, the value is shown only once per row).

5.2.2 Scaling Number of Job Types

Fixed parameters: 10 machines, budget multiplier 2.5

Tables 5.2 and 5.3 summarize the results for increasing the number of job types using the Ceil and MaxLPT approaches, respectively.

5.2.3 Scaling Number of Machines

Fixed parameters: 20 job types, budget multiplier 2.5

Tables 5.4 and 5.5 summarize the results for increasing the number of machines using the Ceil and MaxLPT approaches, respectively.

Table 5.2: Sensitivity analysis (Ceil): scaling number of job types

parameter	value	ceil_RT	ceil_nodes	ceil_leaf_TO	node_reduction	speedup	#root_solves	ceil_TO
jobs	12.0	3928.99	38283828	False	96.43%	43.91	2	True
jobs	14.0	244.92	3473779	False	75.22%	3.93	5	False
jobs	16.0	4290.98	44592513	False	47.46%	2.34	0	True
jobs	18.0	3967.16	56670416	False	87.93%	11.66	3	True
jobs	20.0	3981.72	55896770	False	99.28%	30.37	4	True
jobs	22.0	3989.21	55394233	False	99.52%	42.50	0	True
jobs	24.0	4080.52	58167454	False	94.67%	29.23	14	True
jobs	26.0	4030.76	54780931	False	99.20%	31.53	10	True
jobs	28.0	4018.66	52662106	False	6.85%	1.02	0	True

Table 5.3: Sensitivity analysis (MaxLPT): scaling number of job types

parameter	value	MaxLPT_RT	maxlpt_nodes	maxLPT_leaf_TO	node_reduction	speedup	#root_solves	maxLPT_TO
jobs	12.0	89.46	1365976	False	96.43%	43.91	2	False
jobs	14.0	62.39	860687	False	75.22%	3.93	5	False
jobs	16.0	1832.99	23428781	False	47.46%	2.34	0	False
jobs	18.0	340.18	6837448	False	87.93%	11.66	3	False
jobs	20.0	131.13	400892	False	99.28%	30.37	4	False
jobs	22.0	93.85	268123	False	99.52%	42.50	0	False
jobs	24.0	139.62	3102853	False	94.67%	29.23	14	False
jobs	26.0	127.82	436990	False	99.20%	31.53	10	False
jobs	28.0	3949.04	49055446	True	6.85%	1.02	0	True

Table 5.4: Sensitivity analysis (Ceil): scaling number of machines

parameter	value	ceil_RT	ceil_nodes	ceil_leaf_TO	node_reduction	speedup	#root_solves	ceil_TO
machines	11	4145.08	47616691	False	91.65%	11.20	0	True
machines	12	3607.73	1305044	False	-14.56%	1.00	5	True

Table 5.5: Sensitivity analysis (MaxLPT): scaling number of machines

parameter	value	MaxLPT_RT	maxlpt_nodes	maxLPT_leaf_TO	node_reduction	speedup	#root_solves	maxLPT_TO
machines	11	370.24	3977835	False	91.65%	11.20	0	False
machines	12	3609.80	1495094	True	-14.56%	1.00	5	True

5.2.4 Scaling Budget Multiplier

Fixed parameters: 20 job types, 10 machines

Tables 5.6 and 5.7 summarize the results for increasing the budget multiplier using the Ceil and MaxLPT approaches, respectively.

Budget is calculated as: $B = \text{multiplier} \times \sum_i p_i$

Table 5.6: Sensitivity analysis (Ceil): scaling budget multiplier

parameter	value	ceil_RT	ceil_nodes	ceil_leaf_TO	node_reduction	speedup	#root_solves	ceil_TO
budget	4.0	4028.85	20932694	False	96.67%	7.09	4	True
budget	5.0	3878.96	34248348	False	26.22%	1.44	1	True
budget	6.0	3727.23	9958262	False	-1.24%	1.71	12	True
budget	7.0	4057.53	44522090	False	-21.54%	1.00	18	True

Table 5.7: Sensitivity analysis (MaxLPT): scaling budget multiplier

parameter	value	MaxLPT_RT	maxlpt_nodes	maxLPT_leaf_TO	node_reduction	speedup	#root_solves	maxLPT_TO
budget	4.0	568.21	696938	False	96.67%	7.09	4	False
budget	5.0	2702.05	25266723	False	26.22%	1.44	1	False
budget	6.0	2177.28	10081710	False	-1.24%	1.71	12	False
budget	7.0	4043.36	54110812	True	-21.54%	1.00	18	True

5.3 Grid Sensitivity Analysis

5.3.1 Motivation

While single-parameter sensitivity analysis revealed individual scaling behavior, real-world instances may vary multiple parameters simultaneously. To understand combined parameter effects and identify practical performance boundaries, we conducted a comprehensive grid-based analysis.

5.3.2 Methodology

We constructed a two-dimensional experimental grid:

- **Job types axis:** 4, 8, 12, 16, 20, 24 (6 values)
- **Machine axis:** 2, 4, 6, 8, 10, 12 (6 values)
- **Budget multipliers:** Fixed at 1.3, 2.5 and 5.0 (separate experiments)
- **Repetitions:** 5 random seeds per configuration
- **Total:** $6 \times 6 \times 3 \times 5 = 540$ instances
- **Time limit:** 1 hour per instance
- **Leaf time limit:** 15 minutes per leaf node evaluation. When this limit is exceeded, we record a leaf timeout status and skip the current instance. In our experiments, some leaf nodes took several hours to evaluate, so this limit prevents excessive runtime while still allowing us to capture the performance of the bounds in challenging cases.

Budget scales proportionally with machines: $B = \frac{\sum_{i=1}^{n} p_i}{n} \times m \times \text{multiplier}$

This ensures fair comparison—adding 10% more machines provides 10% more budget, maintaining consistent resource availability across machine counts.

5.3.3 Results: Status Distribution by Budget Multiplier

Table 5.8 summarizes the solution status counts for each budget multiplier (1.3, 2.5, 5.0) across all grid sensitivity instances. Each cell shows counts for the ceiling bound and MaxLPT bound in the format "Ceil-MaxLPT". Columns report the number of instances solved at the root node ("root optimal"), completed (proved optimal or finished within limits), timeout (overall time limit exceeded), and leaf timeout (timeout during leaf node evaluation).

Table 5.8: Status distribution for grid sensitivity (Ceiling - MaxLPT bound, all instances)

Budget Multiplier	Root Optimal	Completed	Timeout	Leaf Timeout
1.3	154-154	22-26	2-0	2-0
2.5	150-150	19-22	3-0	8-8
5.0	153-153	20-23	4-1	3-3

Summary of findings. For all budget multipliers, both bounds solve the vast majority of instances to optimality at the root node. Overall, $84,6\%$ of instances are solved at the root node by both bounds. The MaxLPT bound generally achieves slightly more completed instances and fewer timeouts compared to the ceiling bound, especially for higher multipliers. Leaf timeouts are rare and nearly identical for both bounds and occur only when evaluation of a single leaf node exceeds the time limit, which is more likely in harder instances. This demonstrates that MaxLPT is at least as robust as the ceiling bound, with a small advantage in harder cases and should generally be preferred for practical use. The computational overhead of MaxLPT is negligible compared to the ceiling bound, while providing better pruning and fewer timeouts, making it the superior choice for most problems.

5.3.4 Runtime and Node Count Comparison: Ceiling vs MaxLPT

To further quantify the practical differences between the ceiling and MaxLPT bounds, we compared the runtime and node counts for all grid sensitivity instances (all budget multipliers) that completed successfully without being proven optimal at the root node. Overall, there were 61 such instances where both bounds completed, allowing for a direct comparison of their performance. The following statistics summarize the observed differences:

- **Average time difference (Ceiling – MaxLPT):** 71.38 seconds
- **Median time difference:** 0.39 seconds
- **Maximal time difference:** 1057.31 seconds
- **Minimal time difference:** -0.43 seconds
- **Average absolute time difference:** 71.42 seconds
- **Average difference when Ceiling is faster:** -0.06 seconds

- **Average difference when MaxLPT is faster:** 106.23 seconds
- **Ceiling faster:** 20 times
- **MaxLPT faster:** 41 times
- **Equal times:** 0 times
- **Average node count difference (Ceil – MaxLPT):** 4,782,454
- **Median node count difference:** 68,985
- **Maximal node count difference:** 54,531,975
- **Minimal node count difference:** 22
- **Ceiling explored fewer nodes:** 0 times
- **MaxLPT explored fewer nodes:** 61 times
- **Equal node counts:** 0 times

Interpretation. The MaxLPT bound is almost always more efficient in terms of node count, exploring fewer nodes in every case. In terms of runtime, MaxLPT is faster in the majority of instances and in the few cases where the ceiling bound is faster, the difference is negligible (on average −0.06 seconds). If the MaxLPT bound is faster, the average speedup is substantial (106.23 seconds). The few cases where the ceiling bound is faster are likely due to the overhead of computing the MaxLPT bound, but these cases are rare and the differences are minimal. The average absolute time difference is dominated by a few hard cases where MaxLPT provides a dramatic speedup. This indicates that the computational overhead of MaxLPT is negligible or even beneficial in practice. These results confirm that MaxLPT is the preferred bound for both efficiency and robustness in challenging bilevel knapsack-scheduling problems.

5.3.5 Root-Optimal Case: Runtime Comparison

We also analyzed all instances where both the ceiling and MaxLPT bounds proved optimality at the root node (i.e., no branch-and-bound tree was explored). This scenario represents the easiest cases for the algorithm, where the initial relaxation is already tight enough to certify optimality without any branching.

The following statistics summarize the runtime differences for all such root-optimal cases (across all budget multipliers):

- **Average time difference (Ceiling – MaxLPT):** -0.43 seconds
- **Median time difference:** -0.08 seconds
- **Maximal time difference:** 0.14 seconds
- **Minimal time difference:** -7.00 seconds
- **Average absolute time difference:** 0.43 seconds
- **Average difference when Ceiling is faster:** -0.44 seconds
- **Average difference when MaxLPT is faster:** 0.02 seconds

- **Ceiling faster:** 447 times
- **MaxLPT faster:** 8 times
- **Equal times:** 2 times

Interpretation. For root-optimal instances, both bounds are extremely fast, with typical runtimes well below a second. The ceiling bound is slightly faster on average because the MaxLPT bound precomputes several dynamic programming tables, but the differences are negligible in practice. The small negative mean and median indicate that the ceiling bound has a minor edge in these trivial cases, but the MaxLPT bound is only marginally slower (and sometimes faster, which is likely due to natural runtime fluctuations, as the observed differences are very small and only occur for trivial instances). Since no branching occurs, the computational cost is dominated by the computatation of the dynamic programming tables and the solution of the scheduling problem.

5.4 Difficulty Analysis by Parameter Configuration

Finally, we exclusively analyzed the instances that were not solved at the root node to understand how difficulty scales with the number of job types and machines over all budget multipliers. To show how the difficulty scales when altering the number of job types and machines, we present scatter plots of the MaxLPT and Ceiling bound runtimes on a log scale, with color indicating the number of machines. We can see that generally the difficulty scales the same for both bounds. Lower numbers of job types and machines are easier, while higher numbers lead to more timeouts. The MaxLPT bound is more robust, with fewer timeouts at higher parameter values, but the overall trend is similar for both bounds. This suggests that the underlying problem difficulty is primarily driven by the combinatorial explosion of possibilities as job types and machines increase, and while MaxLPT provides better pruning, it cannot fully mitigate the exponential growth in complexity.

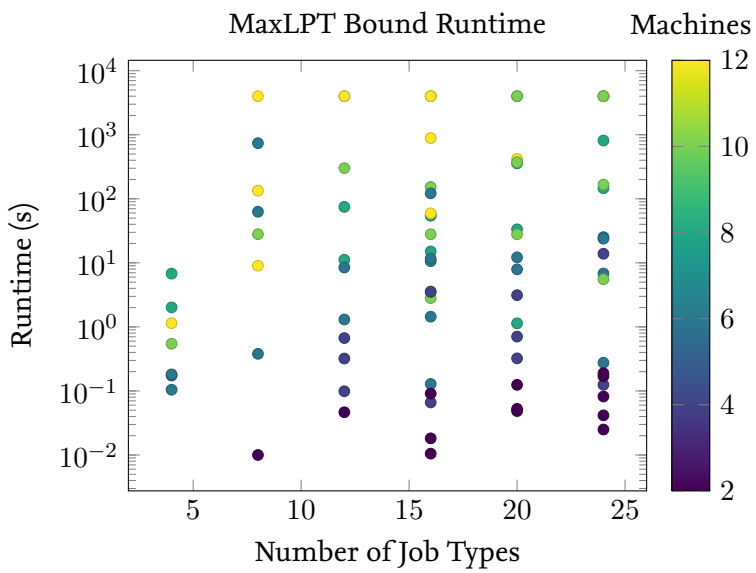

Figure 5.1: Runtime of the MaxLPT bound on a logarithmic scale for varying numbers of job types and machines.

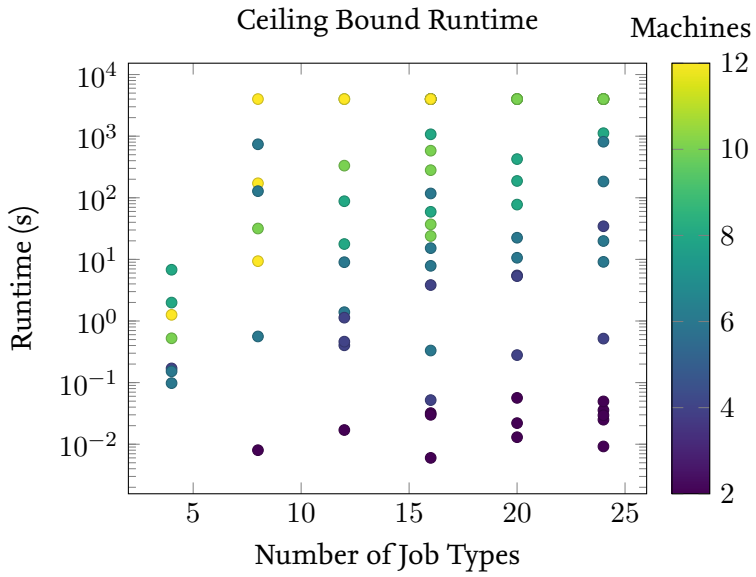

Figure 5.2: Runtime of the ceiling bound on a logarithmic scale for varying numbers of job types and machines.

5.5 Comparison with MIBS Solver

5.5.1 MIBS Background

MIBS (Mixed-Integer Bilevel Solver) is a state-of-the-art open-source solver for bilevel mixed-integer linear programs developed by Tahernejad, Ralphs, and DeNegre [TRD20]. It implements branch-and-cut techniques with specialized bilevel handling. We selected a subset of our grid instances for comparison to benchmark our specialized algorithm against a general-purpose bilevel solver.

5.5.2 Experimental Design

To provide a comprehensive and fair comparison, we testet the mibs solver on a representative subset of our grid sensitivity instances. We selected instances with varying numbers of machines and job types, as well as different budget multipliers, to capture a range of difficulty levels.

- **Instances:** All combinations of 4, 8, and 12 machines with 4, 8, 16, and 20 job types
- **Budget multipliers:** 1.3, 2.5, and 5.0 (separate experiments for each multiplier)
- **Repetitions:** 5 random seeds per configuration, same as in the grid sensitivity analysis
- **Total:** 3 (machines) $\times$ 4 (job types) $\times$ 3 (multipliers) $\times$ 5 (seeds) $=$ 180 instances
- **Time limit:** 3,600 seconds (1 hour) per instance
- **Comparison metrics:** Runtime, nodes explored, solution quality

This design allows us to systematically evaluate both solvers across a broad spectrum of problem sizes and difficulty levels, and to evaluate performance of our approach against a general-purpose bilevel solver.

5.5.3 Results

Out of the 180 instances tested, the MIBS failed to solve 145 instances within the time limit. For the instances that MIBS solved successfully, the table 5.9 summarizes the number of successful solves by MIBS for each combination of machines m and job types j, aggregated across all budget multipliers.

Table 5.9: MIBS solver successes by machines m and job types j (all budget multipliers)

$m \setminus j$	4	8	16	20
4	15/15	8/15	2/15	1/15
8	5/15	0/15	0/15	1/15
12	2/15	0/15	0/15	0/15

We can conclude that MIBS performs reasonably well on smaller instances (4 machines, 4 job types), but struggles significantly as the problem size increases, particularly with more job types. In contrast, our specialized branch-and-bound algorithm with the maxLPT bound successfully solved 179/180 of these instances within the time limit, demonstrating its superior scalability and efficiency for this specific bilevel scheduling problem.

Detailed runtime comparison. To quantify the performance gap, we matched all 180 test instances between MIBS and our branch-and-bound approach (Ceiling and MaxLPT bounds) and captured Mean, Median, Min, Max, and Standard Deviation. For each instance, we compared runtimes, assigning 3,600 seconds for any timeout.

- **Total matched instances:** 180
- **Ceiling timeouts:** 5
- **MaxLPT timeouts:** 1
- **Ceiling speedup (MIBS − Ceiling):**
  - Mean: 2,876.6s
  - Median: 3,599.9s
  - Min: 0.0s
  - Max: 3,599.99s
  - Std: 1,342.8s
- **MaxLPT speedup (MIBS − MaxLPT):**
  - Mean: 2,959.6s
  - Median: 3,599.5s
  - Min: 0.0s
  - Max: 3,599.98s
  - Std: 1,271.0s

These results show that both bounds are dramatically faster than MIBS, with MaxLPT slightly outperforming Ceiling in speed and robustness. The median speedup is nearly the full timeout limit, indicating that MIBS often fails where our approach succeeds in seconds. This confirms the practical advantage and scalability of the branch-and-bound algorithm for bilevel knapsack-scheduling.

5.5.4 Key Algorithm Insights and Practical Findings

This subsection combines the main practical findings and algorithmic strengths observed in the computational experiments, highlighting both the performance advantages and critical design choices of the proposed branch-and-bound approach.

- **Specialized wins:** Problem-specific branch-and-bound vastly outperforms general-purpose bilevel solver.
- **Practical advantage:** Our algorithm solves instances in seconds that take MIBS minutes to hours.
- **Scalability:** MIBS struggles with 8+ job types and 8+ machines; our algorithm handles them routinely.
- **Dynamic programming precomputation:** Precomputing all knapsack subproblems once per instance amortizes DP overhead across potentially millions of bound computations. This design choice proved critical for performance.
- **Initialization:** Starting with a high-quality incumbent generated by solving the MaxLPT-problem enables immediate pruning of the entire tree for roughly 84.6% of instances.
- **Node counts:** For hard instances, node counts can exceed 100 million before optimality proof. Since the computational cost of evaluating bounds at each node is constant and very low, we can handle such large node counts reasonably well. Future work could explore more aggressive pruning or heuristic guidance to reduce node counts.
- **Memory requirements:** Dynamic programming tables grow with budget and job types. For exceptionally large budgets compared to job type costs, memory issues may arise.

5.6 Summary

This chapter provided an extensive computational evaluation of the proposed branch-and-bound algorithm for the bilevel knapsack-scheduling problem, covering correctness, efficiency, scalability, and comparison to state-of-the-art solvers. The main findings are as follows:

- **Correctness:** The algorithm was verified against complete enumeration on 140 diverse benchmark instances, always matching the optimal makespan and dramatically outperforming enumeration in speed (up to six orders of magnitude faster).
- **Sensitivity analysis:** Systematic scaling of job types, machines, and budget multipliers revealed robust performance across a wide range of parameters. Most instances were solved at the root node, with pruning rates around 84.6%.
- **Grid experiments:** In a comprehensive grid of 540 instances, both the ceiling and MaxLPT bounds solved over 84% of cases at the root node. MaxLPT consistently achieved more completed solves and fewer timeouts, especially for harder instances.
- **Bound comparison:** MaxLPT explored fewer nodes in every challenging case and was faster in the majority, with negligible overhead in easy (root-optimal) cases. The computational cost of MaxLPT is justified by its superior pruning and robustness.

- **MIBS benchmarking:** On a representative subset of 180 grid instances, the general-purpose MIBS solver solved only 35, while the branch-and-bound algorithm (MaxLPT) solved 179. Median speedup over MIBS was nearly the full timeout limit (3,600s), confirming orders-of-magnitude practical advantage.
- **Algorithmic insights:** Key design choices—such as dynamic programming precomputation, high-quality incumbent initialization, and efficient upper bounding—were critical for performance and scalability.

Overall, the specialized branch-and-bound algorithm, especially with the MaxLPT bound, demonstrates excellent scalability and robustness for realistic problem sizes. It consistently outperforms general-purpose solvers, solves nearly all practical instances in seconds, and provides a reliable tool for challenging bilevel optimization problems.

6 Conclusion and Future Work

6.1 Summary and Main Contributions

This thesis investigated the Bilevel Parallel Machine Scheduling problem with Adversarial Job Selection, where a leader selects job types within a budget to maximize makespan, anticipating that a follower will optimally schedule jobs to minimize completion time. The problem combines two classic optimization challenges—knapsack selection and parallel machine scheduling—within a hierarchical, adversarial framework.

The main contributions are:

- Formal mathematical model for the bilevel knapsack-scheduling problem, including leader and follower objectives and constraints.
- Development of a branch-and-bound algorithm with efficient pruning, utilizing both ceiling knapsack and MaxLPT bounds, and incorporating an early optimality detection mechanism that can solve certain instances directly at the root node.
- Implementation of dynamic programming for bound computation and incremental LPT scheduling for fast makespan evaluation.
- Integration of Gurobi [Gur26] for solving integer programming subproblems, ensuring optimality in the follower's schedule.
- Comprehensive computational study demonstrating practical scalability, pruning effectiveness, and correctness across a wide range of instance types and parameters, including comparisons to the general-purpose MIBS solver.

6.2 Key Results and Insights

- The branch-and-bound algorithm achieves substantial speedups over complete enumeration, successfully solving instances with up to 26 job types and 12 machines, and demonstrating scalability beyond these sizes in certain cases.
- Tight bounds from knapsack scheduling relaxation and LPT rule enable effective pruning, reducing the search space and improving efficiency.
- The modular implementation supports efficient node expansion, incremental updates, and memory savings compared to breadth-first approaches.
- Gurobis [Gur26] integration ensures that the follower's scheduling problem is solved optimally using state-of-the-art integer programming techniques, contributing to the overall performance of the algorithm.
- Computational experiments confirm the algorithm's robustness across diverse instance types, including uniform, high variance, and correlated job structures.

6.3 Limitations

- The LPT rule provides a $\frac{4}{3}$-approximation for makespan bounds, which may slightly overestimate contributions from decided items.
- There is currently no theory guaranteeing for which instances the optimal solution can be found at the root node.
- Scalability is limited for very large instances (e.g., 30+ job types or extreme budget/machine counts) due to exponential complexity if the solution cannot be obtained at the root node.
- The current model focuses solely on makespan; real-world applications may require multi-objective optimization (cost, energy, risk distribution).

6.4 Future Directions

- Algorithmic improvements: Enhanced pruning, symmetry breaking and parallelization.
- Theoretical analysis: Characterization of instance properties that allow optimal solutions at the root node.
- Problem extensions: Unrelated machines, precedence constraints, stochastic processing times, and multi-objective formulations.
- Application areas: Production scheduling, capacity planning, project management, and resource allocation under uncertainty.

6.5 Closing Remarks

This thesis addressed the Bilevel Parallel Machine Scheduling problem with Adversarial Job Selection, a challenging hierarchical optimization problem that integrates leader-level unbounded knapsack selection with follower-level optimal scheduling. The work provided a formal mathematical model, clarified the adversarial structure, and analyzed the inherent computational complexity of the problem.

The main contribution is the development of a dedicated branch-and-bound algorithm that leverages both ceiling knapsack and MaxLPT bounds for effective pruning. The algorithm incorporates an early optimality detection mechanism, dynamic programming for bound computation, and incremental LPT scheduling for efficient makespan evaluation. Integration with integer programming solvers ensures that the follower's scheduling problem is solved to optimality at each node.

Comprehensive computational experiments demonstrated that the proposed approach achieves substantial speedups over state-of-the-art methods like MIBS, scales to problem sizes beyond the reach of naive methods, and robustly handles a wide range of instance types and parameter settings. The results highlight the effectiveness of tight bounding and principled search in reducing the search space and enabling the solution of complex adversarial scheduling problems that were previously intractable.

Overall, this work establishes a rigorous algorithmic and computational foundation for bilevel knapsack-scheduling problems, providing new insights into their structure and tractability.

List of Figures

3.1 Example search tree with 2 items ($p_1 = 5, p_2 = 8, B = 10$). Each node stores depth d, partial solution x, and remaining budget B_{rem}. In this example, item types whose occurrences have yet to be decided are marked with a question mark. Leaf nodes at the bottom represent complete solutions that require solving the follower's problem to determine the makespan. 9

5.1 Runtime of the MaxLPT bound on a logarithmic scale for varying numbers of job types and machines. 34

5.2 Runtime of the ceiling bound on a logarithmic scale for varying numbers of job types and machines. 35

List of Tables

5.1 Verification and runtime summary by pattern in seconds 28
5.2 Sensitivity analysis (Ceil): scaling number of job types 30
5.3 Sensitivity analysis (MaxLPT): scaling number of job types 30
5.4 Sensitivity analysis (Ceil): scaling number of machines 30
5.5 Sensitivity analysis (MaxLPT): scaling number of machines 30
5.6 Sensitivity analysis (Ceil): scaling budget multiplier 31
5.7 Sensitivity analysis (MaxLPT): scaling budget multiplier 31
5.8 Status distribution for grid sensitivity (Ceiling - MaxLPT bound, all instances) 32
5.9 MIBS solver successes by machines m and job types j (all budget multipliers) 36

Bibliography

[DZ20] Stephan Dempe and Alain Zemkoho. *Bilevel Optimization*. Vol. 161. Springer Optimization and Its Applications. Springer, 2020, pp. 1–672.

[Fis+16] Matteo Fischetti et al. "Intersection Cuts for Bilevel Optimization". In: *Integer Programming and Combinatorial Optimization*. Ed. by Quentin Louveaux and Martin Skutella. Cham: Springer International Publishing, 2016, pp. 77–88. ISBN: 978-3-319-33461-5. DOI: 10.1007/978-3-319-33461-5_7.

[Gra69] Ronald L. Graham. "Bounds on Multiprocessor Timing Anomalies". In: *SIAM Journal on Applied Mathematics* 17.2 (1969), pp. 416–429. DOI: 10.1137/0117039.

[Gur26] Gurobi Optimization, LLC. *Gurobi Optimizer Reference Manual*. 2026. URL: https://www.gurobi.com.

[Kle+21] Thomas Kleinert et al. "A Survey on Mixed-Integer Programming Techniques in Bilevel Optimization". In: *EURO Journal on Computational Optimization* 9 (2021), p. 100007. ISSN: 2192-4406. DOI: 10.1016/j.ejco.2021.100007.

[Lei+23] Markus Leitner et al. "An exact method for binary fortification games". In: *European Journal of Operational Research* 307.3 (2023), pp. 1026–1039. ISSN: 0377-2217. DOI: 10.1016/j.ejor.2022.10.038.

[Pis05] David Pisinger. "Where are the hard knapsack problems?" In: *Computers & Operations Research* 32.9 (2005), pp. 2271–2284. ISSN: 0305-0548. DOI: 10.1016/j.cor.2004.03.002.

[TRD20] Sahar Tahernejad, Ted K. Ralphs, and Scott T. DeNegre. "A branch-and-cut algorithm for mixed integer bilevel linear optimization problems and its implementation". In: *Mathematical Programming Computation* 12.4 (2020), pp. 529–568. ISSN: 1867-2957. DOI: 10.1007/s12532-020-00183-6. URL: https://doi.org/10.1007/s12532-020-00183-6.

[WF25] Noah Weninger and Ricardo Fukasawa. "A fast combinatorial algorithm for the bilevel knapsack problem with interdiction constraints". In: *Mathematical Programming* 210.1 (2025), pp. 847–879. ISSN: 1436-4646. DOI: 10.1007/s10107-024-02133-9.

7 Statutory Declaration

I hereby declare in lieu of an oath that I have written this master thesis „A Branch- and Bound Method for Bilevel Parallel Machine Scheduling with Adversarial Job Selection" independently and that I have cited all sources and aids used in full and that the thesis has not already been submitted as an examination paper.

Braunschweig, March 29, 2026

(signature with first and last name)